



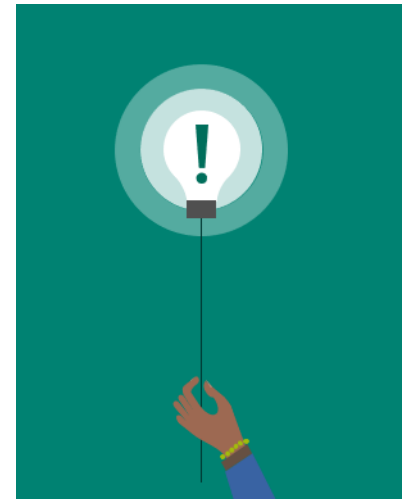
Error Handling

Module 5

Objectives

Learnings covered:

- Introduction to Error Handling
- Streams
- Write-Host and Streams
- Working with ErrorRecord Object
- Non-Terminating Errors
- Introduction to Terminating Errors
- Catching Terminating Errors using Trap
- Catching Terminating Errors Try {} Catch {} Finally {}



Introduction to Error Handling

What is an error?

PowerShell representation of an object-based exception

Design Time Errors

Syntax errors are caught by the PowerShell parser and/or PowerShell ISE

Runtime Errors

When something goes wrong during code execution

Only detectable when a specific state is reached, or statement is executed

Types of errors

Cmdlet/Function/Script/Object author decides which type of errors to trigger

Terminating

- Stops a statement from running
- If PowerShell does not handle the terminating error, it stops running the function or script

Non-Terminating

- Less severe, often just a single operation out of many
- Primary processing doesn't need to stop
- Error may be displayed in host application

What to do when an error occurs?

Nothing

- Red error messages are displayed
- Results could be unpredictable

Debug Code

- Identify where the error has occurred
- Resolve syntax or logic problems

Handle the Errors with Code

- Typically hide the red error messages
- Write logic to handle errors appropriately
- Actions can be: ignore, process, log, raise, or halt further execution

Error Handling: Terminating vs. Non-Terminating

Terminating Errors:

- Requires Try, Catch, Finally or a Trap statement
- Enters a child scope

Non-Terminating Errors:

- Test for error and run handling code
- Can be converted into terminating error
 - "Stop" with `-ErrorAction` or `$ErrorActionPreference`
 - Use the "throw" keyword

Questions?

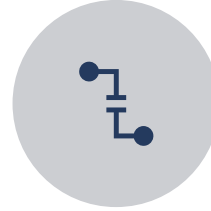


Streams

Understanding Streams



Streams separate different categories of output.



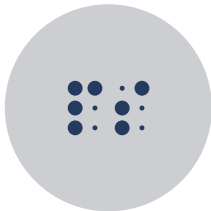
Without separation, error and output objects would be mixed, requiring later filtering.



Some streams are hidden by default (Verbose, Debug, Information).



Redirection operators allow control over a stream's destination.



Write-* sends object to related streams.

The Streams

1 - Output



- Most Common
- Flows down pipeline
- Captured by Assignment '='

2 - Error



- Execution Problems.
- Shown by default in red

3 - Warning



- Less severe execution problems
- Shown by default in yellow

4 - Verbose



- More detailed execution problems
- Hidden by default.

5 - Debug



- Related to debugging code
- Hidden by default

6 - Information



- Better for logging Introduced in PowerShell 5
- Hidden by default

Stream Cmdlets

```
PS C:\> "Output Stream"
Output Stream
```

```
PS C:\> Write-Output "Output Stream"
Output Stream
```

```
PS C:\> Write-Error "Error Stream"
Write-Error "Error Stream" : Error Stream
    + CategoryInfo          : NotSpecified: (:) [Write-Error], WriteErrorException
    + FullyQualifiedErrorId : Microsoft.PowerShell.Commands.WriteErrorException
```

```
PS C:\> Write-Warning "Warning Stream"
WARNING: Warning Stream
```

```
PS C:\> Write-Verbose "Verbose Stream" -Verbose
VERBOSE: Verbose Stream
```

```
PS C:\> Write-Debug "Debug Stream" -Debug
DEBUG: Debug Stream
```

```
PS C:\> Write-Information "Information Stream" -InformationAction Continue
Information Stream
```

Controlling Stream Behavior

Preference Variables

- Variables that control how PowerShell reacts to stream messages

Cmdlet common parameters

- -Verbose, -Debug, -ErrorAction, -WarningAction

No preference variable for Output Stream

- Controlled by assignments, redirection, and pipeline

Stream Defaults

```
PS C:\> $ErrorActionPreference  
Continue
```

```
PS C:\> $WarningPreference  
Continue
```

```
PS C:\> $VerbosePreference  
SilentlyContinue
```

```
PS C:\> $DebugPreference  
SilentlyContinue
```

```
PS C:\> $InformationPreference  
SilentlyContinue
```

Redirecting Streams to Files (Output, Error, Warning)

Operator	Description
>	Sends output to specified file
> >	Appends output to contents of specified file
2>	Sends errors to specified file
2> >	Appends errors to contents of specified file
2> &1	Sends errors (2) and success output (1) to success output stream
3>	Sends warnings to specified file
3> >	Appends warnings to contents of specified file
3> &1	Sends warnings (3) and success output (1) to success output stream

Redirecting Streams to Files (Verbose, Debug)

Operator	Description
4>	Sends verbose output to specified file
4> >	Appends verbose output to contents of specified file
4> &1	Sends verbose output (4) and success output (1) to success output stream
5>	Sends debug messages to specified file
5> >	Appends debug messages to contents of specified file
5> &1	Sends debug messages (5) and success output (1) to success output stream

Redirect everything

Operator	Description
6>&1	Sends information messages (5) and success output (1) to success output stream
*>	Sends all output types to specified file
*>>	Appends all output types to contents of specified file
>&1	Sends all output types () to success output stream

Write-Host and Streams

- Until PowerShell 5 **Write-Host** output **does not go to a stream** but instead to **console** therefore:
 - Cannot suppress Write-Host output
 - Cannot save the result into a variable (assignment operator uses the Output stream)
 - Cannot be redirected since its streamless
- Being streamless was also a benefit
 - Helps in isolating the object returned and the Information
- PowerShell 5 solves the above dilemma with the **Information stream**
- **Write-Host** is now a **wrapper** for **Write-Information**
 - Write-Host output is now in Information Stream and is isolated from Object output!

Demonstration

Streams



Questions?



Write-Host and Streams

Issues with Write-Host

Until PowerShell 5

- Write-Host output does not go to a stream but instead to console.
- Could not suppress Write-Host output
- Could not save the result into a variable (assignment operator uses the Output stream)
- Cannot be redirected since it's streamless

Benefits of Write-Host

Until PowerShell 5

- Being streamless helps in **isolating the object** returned and the Information
 - Prevents output stream pollution
 - Prevents pipeline processing (better pipeline performance)
- Stream isolation is **partially** achieved

Solution with PowerShell 5

- Information Stream introduced
- Write-Information Cmdlet supports,
 - \$InformationVariable
 - Redirection
 - Suppression through **\$InformationPreference** variable and **-InformationAction** parameter
 - InformationRecord object is sent to Information stream
- Write-Host is now a wrapper for Write-Information
 - Can now redirect

Write-Host vs Write-Information

Feature	Write-Information	Write-Host
\$InformationPreference & -InformationAction	😊	😞
Colorful	😞	😊
Has Tag Parameter	😊	😞
Takes value from pipeline	😞	😊
Output can be redirected	😊	😊

Note: both have the **InformationAction** parameter but **Write-Host** just does not respond

Best Practices

Upgrade to at least
WMF 5 (PowerShell 5)

Both Cmdlets isolate
information from
output stream

For **Scripting &
Logging** purposes use
Write-Information

- Can be suppressed
- Supports tagging

Use **Write-Host** for
Console output &
Colorful Host
Information

Demonstration

Write-Host and Streams



Questions?



Working with ErrorRecord Object

ErrorRecord Object



Is stored in and accessed via **\$Error** array automatic variable



Is an instance of ErrorRecord Class in **System.Management.Automation** Namespace



Is an **extension** to **System.Exception** .Net class with PowerShell with additional PowerShell related properties



The actual System.Exception is stored in **Exception property** of the ErrorRecord Class



The PowerShell host investigates ErrorRecord Objects and determines to stop or not the pipeline

\$Error – An array of ErrorRecord objects

- Automatic variable – Array holds errors that have occurred in the current session
- Control maximum array size:

```
PS C:\> $MaximumErrorCount  
256
```

- The most recent error is the first object in the array:

```
PS C:\> $Error[0]
```

- Prevent an error from being added to the \$Error array by using "-ErrorAction Ignore"

\$Error – View all properties of ErrorRecord

- By default **\$Error** only displays a summary of the error
- View all properties using one of the following examples:



```
PS C:\> $Error[0] | Format-List * -Force
PS C:\> $Error[0] | fl * -Force
PS C:\> $Error[0] | select-object *
PS C:\> $Error[0] | select *
```

- Drill into the object model for more error details

```
PS C:\> $Error[0].CategoryInfo
PS C:\> $Error[0].InvocationInfo
PS C:\> $Error[0].ScriptStackTrace
```

\$? – Last operation execution status

Automatic Variable

- Contains execution status of **last operation**
- Applies to both terminating and non-terminating errors
- Even applies to **external command** exit codes
- True = Complete Success
- False = Failure (Partial or Complete)
- Typically used in an `if()` statement to test and then run error handling code

-ErrorVariable Common Parameter

Capture errors from a specific action into a dedicated variable

Create the variable and store any errors in it:

```
Get-Process -Id 6 -ErrorVariable MyErrors
```

Append error messages to the variable:

```
Get-Process -Id 6 -ErrorVariable +MyErrors
```

Work with the variable just like \$Error:

```
$MyErrors
```

\$Error holds **all errors**, including those sent to **-ErrorVariable**.

Demonstration

Working with ErrorRecord
Object



Questions?



Non-Terminating Errors

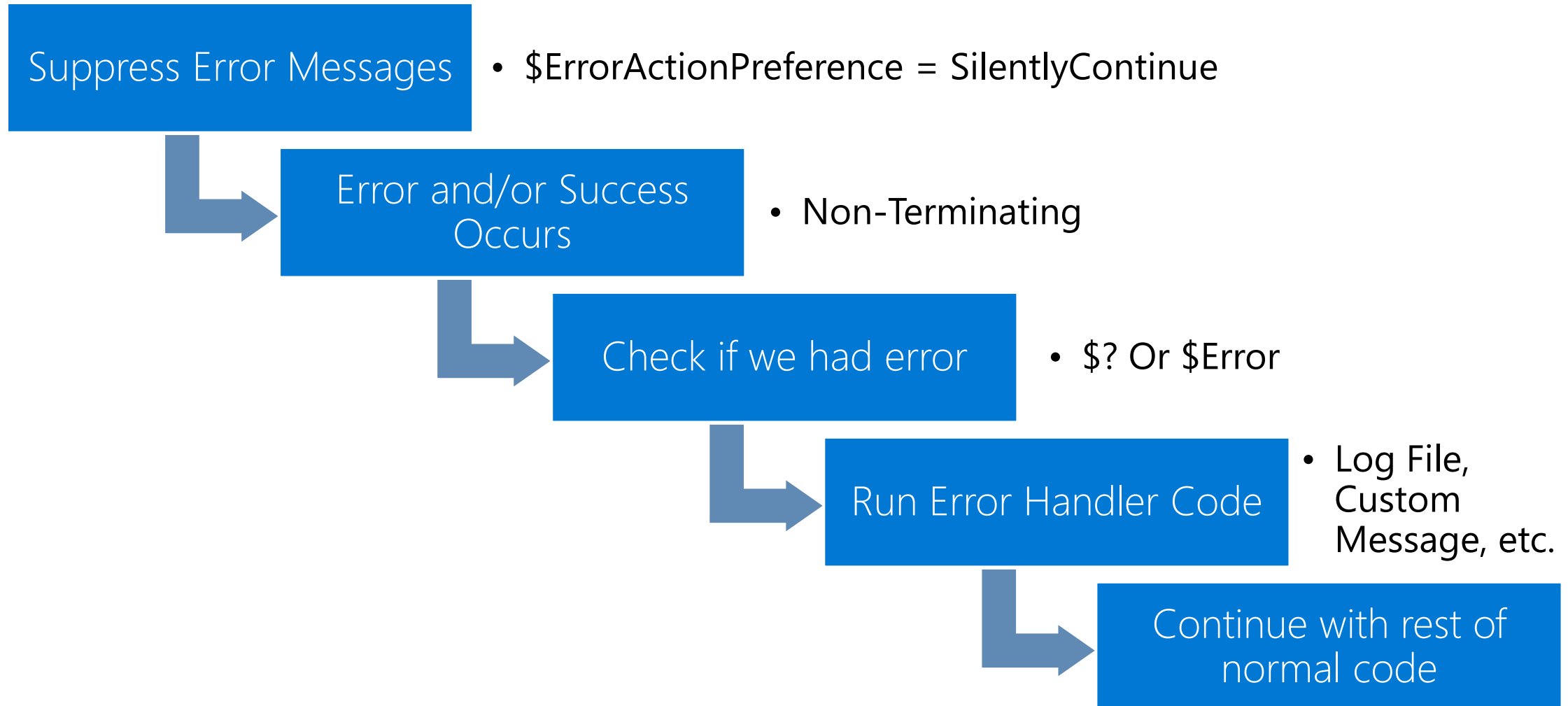
Errors do not stop processing

- Optionally handle error in code
- Continue with rest of code
- Does not trigger Trap, Try..Catch..Finally

Can be converted into Terminating errors

- -ErrorAction Stop or
\$ErrorActionPreference = 'Stop'
- Can then utilize Trap, Try..Catch..Finally

Non-Terminating error handling flow



Actions to Suppress Error Messages

`$ErrorActionPreference`

- Automatic variable for remaining script/function/session
- Affects all the Cmdlets and Advanced Functions in Current scope

`-ErrorAction` common parameter

- On all cmdlets and advanced functions/scripts
- Affects only command where used
- Use `SilentlyContinue` value to suppress Error Messages

-ErrorAction Values

Determines PowerShell response to non-terminating errors

Optional Numerical Values		Continue (Default)	Displays error message and continues executing
		SilentlyContinue	Error message is not displayed, and execution continues w/o interruption
		Stop	Raises terminating error and displays an error message and stops command execution
		Suspend	- Automatically suspends a workflow job - Allows for investigation, can be resumed
		Inquire	Displays error message and prompts user to continue
		Ignore	Can only be used with -ErrorAction common parameter (not with \$ErrorActionPreference)
SilentlyContinue	0		
Stop	1		
Continue	2		
Inquire	3		
Ignore	4		
Suspend	5		

\$ErrorActionPreference

Variable value affects all subsequent commands in same variable scope

- Errors are displayed by default:

```
PS C:\> $ErrorActionPreference  
Continue
```

- Visible Errors Suppressed:

```
PS C:\> $ErrorActionPreference = "SilentlyContinue"  
# or  
PS C:\> $ErrorActionPreference = 0
```


Write-Error

- Creates non-terminating errors
- Can be handled in-code via \$?
- Automatically stored in \$Error
- Useful for re-usable functions to report errors to caller



```
PS C:\> Write-Error "My Custom Error"
Write-Error "My Custom Error" : My Custom Error
+ CategoryInfo          : NotSpecified: (:) [Write-Error], WriteErrorException
+ FullyQualifiedErrorId : Microsoft.PowerShell.Commands.WriteErrorException

PS C:\> $Error[0]
Write-Error "My Custom Error" : My Custom Error
+ CategoryInfo          : NotSpecified: (:) [Write-Error], WriteErrorException
+ FullyQualifiedErrorId : Microsoft.PowerShell.Commands.WriteErrorException
```

Demonstration

Non-Terminating Errors



Questions?



Introduction to Terminating Errors

Terminating Errors



Severe errors where processing of the current scope can't continue



Error cannot be handled in the same code block, like non-terminating errors



Use `Trap` or `Try{} Catch{} Finally{} to handle`



Non-Terminating Errors can be re-thrown as terminating errors using
"-ErrorAction stop"

Overview of Trap/Try..Catch..Finally

Trap

Learn the Type of
the Exception



Set a trap for the Type



Run CODE



Terminating Error
Triggers the trap code

Try/Catch/Finally

Learn the Type of
the Exception



Encapsulate CODE into
Try Block



Use the Type in Catch
Block



Finally block always
runs even during Ctrl-C

Type of the Exception

Both Try{} Catch{} Finally{} Catches the Type (Class) of the Exception Object

It can be specific or generic. Common Exception Types;

- SystemException
- IndexOutOfRangeException
- NullReferenceException
- InvalidOperationException
- ArgumentException
- ArgumentNullException
- Argument out of range exception

Common

Specific

To learn the current ErrorRecords ExceptionType, use FullName;

```
PS C:\> $Error[0].Exception.InnerException.GetType() | select-object -  
Property Name, BaseType, FullName | fl
```

```
Name      : DivideByZeroException  
BaseType  : System.ArithmeticException  
FullName  : System.DivideByZeroException
```

Throw

- User-created terminating error.
- Throws a message string or any object type.
- Can be used to enforce mandatory parameters
 - PowerShell 3.0+, use the [Parameter(Mandatory)] attribute
- Useful in re-usable code to cause halt and report errors to caller for severe errors

```
PS C:\> If ( -not (Test-Path $UserFile)) { Throw "ERROR: File not found" }
```


Example: Trace parameter binding

Basic Throw



```
If (1 -eq 1)
{
    "Line before the terminating error"
    Throw "This is my custom terminating error"
    "Line after the throw"
}
```

Line after throw doesn't
run because of termination

```
PS C:\> C:\Simple-Throw-Sample.ps1
Line before the terminating error
This is my custom terminating error
At C:\Users\danpark\OneDrive @ Microsoft\Scripting\Simple-Throw-Sample.ps1:4
char:5
+      Throw "This is my custom terminating error"
+      ~~~~~
+ CategoryInfo          : OperationStopped: (This is my custom terminating
error:String) [], RuntimeException
+ FullyQualifiedErrorId : This is my custom terminating error

PS C:\>
```

Demonstration

Terminating Errors



Questions?



Catching Terminating Errors using Trap

Trap { }



Specifies a list of statements to run when a terminating error occurs



Handles terminating errors and allow execution to continue



Can catch all, generic (namespace), or specific exception types



Multiple Trap statements can be specified in a script



Is "global", applies to all code in the same scope, before or after

Example: Trap

```
Trap [System.DivideByZeroException] {    #This is specific trap
    "Can not divide by ZERO!!!-->    " + $_.exception.message
    Continue
}
Trap {#This is a generic catch all trap
    "A serious error occurred-->    " + $_.exception.message
    Continue
}
Write-Host -ForegroundColor Yellow "`nAttempting to Divide by Zero"
1 / $null
Write-Host -ForegroundColor Yellow "`nAttempting to find a fake file"
Get-Item c:\fakefile.txt -ErrorAction Stop
Write-Host -ForegroundColor Yellow "`nAttempting an invalid command"
1..10 | ForEach-Object {Bogus-Command}
```

Attempting to Divide by Zero

Can not divide by ZERO!!!--> Attempted to divide by zero.

Attempting to find a fake file

A serious error occurred--> Cannot find path 'C:\fakefile.txt' because it does not exist.

Attempting an invalid command

A serious error occurred--> The term 'Bogus-Command' is not recognized as the name of a cmdlet, function, script file, or operable program. Check the spelling of the name, or if a path was included, verify that the path is correct and try again.

Demonstration

Catching Terminating Errors
using Trap



Catching Terminating Errors Try {} Catch {}
Finally {}

Try, Catch, Finally Syntax

```
Try
{
    <Code that could cause terminating errors>
}

Catch [Exception Type1], [Exception Type2], [Exception Type3]
{
    <Error handling>
}

Catch
{
    <Handle uncaught errors>
}

Finally
{
    <Clean up code>
}
```



- One or more catch blocks
- Exception type optional
Note: most specific to least specific
- Finally block optional
- Must have at least one Catch or Finally block

Try, Catch, Finally Flow

The Try block defines a section of a script to monitor for errors



When an error occurs within the Try block, the error is first saved to \$Error



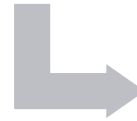
PowerShell then searches for a Catch block to handle the error



If no matching Catch block is found or defined, then parent scopes are searched



After the Catch block, the Finally block executes



If the error cannot be handled, the error is written to the error stream

Finally Block

Will run even if Ctrl-C is used during the Try block execution

Useful for:

- Releasing resources
- Closing network connections
- Closing database connections
- Logging
- Etc.

Example: Try/Catch/Finally

```
Try
{
    $wc = New-Object System.Net.WebClient
    $wc.DownloadData("http://www.contoso.com/MyDoc.doc")
}
Catch [System.Net.WebException],[System.IO.IOException]
{
    "Cannot get MyDoc.doc from http://www.contoso.com"
}
Catch
{
    "An error occurred that could not be resolved."
    $_.Exception.Message
}
Finally
{
    If ($wc)
    {
        $wc.Dispose()
    }
}
```



\$_ contains caught error

Scopes and Try, Catch, Finally

Exceptions can be re-thrown to the parent scope

- i.e. from a function to the calling scope (exception 'bubbling')
- Unhandled exceptions in any scope will be processed by the Windows PowerShell Host

In the code below

- The "function3" catch block is executed, which throws a new exception
- The new exception is caught by the parent catch block
- This affects flow control, as the "function3 was completed" text is NOT executed

```
$ErrorActionPreference = SilentlyContinue

Function function3 {
    Try {NonsenseCommand}
    Catch {"Error trapped inside function" ; Throw}
    "Function3 was completed"
}

Try {Function3}
Catch { "Internal Function error re-thrown: $($_.ScriptStackTrace)" }
"Script Completed"
```

Demonstration

Catching Terminating Errors
using Try{} Catch{} Finally{}



Questions?



Error Handling

